

114. Le hasard dans une plage choisie

`hasard()` rend 0 à 255 ; « début + `hasard()` % combien » le range entre deux bornes.

1. Ce qu'il faut comprendre

`hasard()` rend un nombre imprévisible de **0 à 255**. On ne veut presque jamais ça : on veut un dé de 1 à 6, une colonne de 2 à 17, une chance sur quatre.

Ce qui est nouveau ici : la formule `debut + hasard() % combien`. Le modulo (chapitre 1) rend un reste **de 0 à combien - 1** ; on ajoute le début. Pour un dé : `1 + hasard() % 6` → reste de 0 à 5, plus 1 → **1 à 6**. Pour une colonne de 2 à 17 : il y a 16 colonnes, donc `2 + hasard() % 16` → **2 à 17**.

Exemples chiffrés : si `hasard()` rend 200, $200 \% 6 = 2$ (car $6 \times 33 = 198$, reste 2) → face 3. S'il rend 17 : $17 \% 6 = 5$ → face 6. S'il rend 0 : face 1.

Un appui, un tirage. `bouton(A)` reste vrai **tant que** A est enfoncé : plusieurs images de suite, donc plusieurs tirages. On veut un tirage **au moment où on appuie**. `avant` retient l'état de A à l'image d'avant ; `a && !avant` n'est vrai qu'à la première image de l'appui (enfoncé maintenant, relâché avant). On appelle cela un **front**.

Un détail honnête : 256 n'est pas divisible par 6 ($256 = 6 \times 42 + 4$). Les faces 1 à 4 sortent donc 43 fois sur 256, les faces 5 et 6 seulement 42 fois. La différence est minuscule pour un jeu ; elle compterait pour un casino.

À toi : tire une chance sur quatre (`hasard() % 4 == 0`) et affiche « BONUS! » quand elle tombe.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **897 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	69 o	← la console, à l'allumage — la mise en route de la console
main()	168 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d'une division

CopierTuiles	103 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Hasard	25 o	← hasard()	– tire un nombre au hasard
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	320 o	← CopierTuiles	– 20 lettres sur 43
"A: LANCER"	9 o	← texte()	– un texte gravé
"DE"	2 o	← texte()	– un texte gravé
" ..."	20 o	← texte()	– un texte gravé
"P"	1 o	← texte()	– un texte gravé
" "	1 o	← texte()	– un texte gravé
"p"	1 o	← texte()	– un texte gravé

Lettres gravées : ACDELNPR0123456789:|

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 6	87 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
nombre()	× 2	20 o sur place, dans main()	+ EcrireNombre (33 o)
bouton()	× 1	12 o sur place, dans main()	+ LireManette (37 o)
hasard()	× 2	6 o sur place, dans main()	+ Hasard (25 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : chaque appui sur A lance un dé (1 à 6) et pose un P
// à une place au hasard sur une piste (colonnes 2 à 17).
//
// Ce qui est nouveau : le hasard DANS UNE PLAGES choisie.
//  hasard() rend un nombre de 0 à 255. La formule :
//  début + hasard() % combien → un nombre de début à début + combien - 1
//  Un dé : 1 + hasard() % 6 → reste de 0 à 5, plus 1 → 1 à 6.
//  Exemple : hasard() rend 200 → 200 % 6 = 2 (6 × 33 = 198, reste 2) → face 3.

#include <texte>    // écrit un texte à l'écran
#include <nombre>  // écrit un nombre en chiffres
#include <bouton>   // lit un bouton de la manette
#include <hasard>  // tire un nombre au hasard
#include <reste>   // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

2. UNE VARIABLE — DE

```
uint8_t de = 1;    // le dernier dé tiré : 1 à 6
```

3. UNE VARIABLE — COLONNE

```
uint8_t colonne = 2; // la dernière colonne tirée : 2 à 17
```

4. UNE VARIABLE — AVANT

```
uint8_t avant = 0;   // A était-il enfoncé à l'image d'avant ?
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 1, "A: LANCER");
    texte(1, 4, "DE");
    texte(1, 8, "|");                       // la piste : colonnes 1 et 18 sont les bords
    nombre(5, 4, de, 1);                   // Le dé, sur 1 chiffre.
    texte(colonne, 8, "P");                 // Le P à sa place de départ.

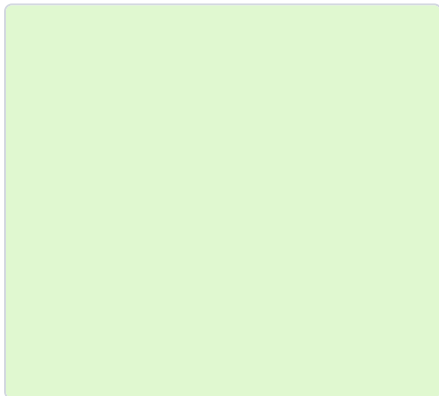
    while (true) {                          // La boucle du jeu :
        image();

        uint8_t a = bouton(A);              // A, maintenant
        if (a && !avant) {                  // enfoncé maintenant, pas avant : un FRONT
                                            // (un seul tirage par appui, même si on garde le doigt)
            de = 1 + hasard() % 6;          // reste 0 à 5, plus 1 : 1 à 6
            nombre(5, 4, de, 1);

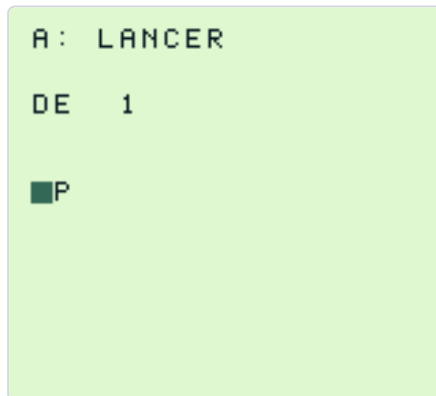
            texte(colonne, 8, " ");         // efface l'ancien P
            colonne = 2 + hasard() % 16;    // reste 0 à 15, plus 2 : 2 à 17
            texte(colonne, 8, "P");         // dessine le nouveau
        }
        avant = a;                          // on retient A pour l'image suivante
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



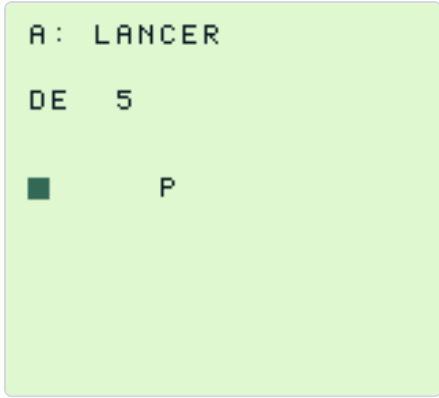
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Un dé et un P sur une piste ; chaque appui sur A tire une face de 1 à 6 et une nouvelle place pour le P, toujours entre les deux bords.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ chaque dé est entre 1 et 6 (1 2 3 4 5 6)
- ✓ plusieurs faces sortent
- ✓ chaque colonne est entre 2 et 17
- ✓ un seul P sur la piste, à sa colonne

Ensuite : **115. Une place libre : tirer encore** — chapitre 19 : Le hasard et le temps. · gameboy3 — écrire une cartouche Game Boy en C++